SmartScheduler — Final Project Proposal & Technical Specification

CSS 382 · Design Your Own Project (DYOP)
University of Washington Bothell
Team (3 members):

- **Richie Chen** — Backend / Data Pipeline (scraper, catalog parser, embedding & vector store, FastAPI)
- **Kevin Vo** — LLM / RAG Integration (Gemini File Search RAG, prompt engineering, conflict-detection logic)
- **Yousuf Al-Bassyiouni** — Frontend / UX (React SPA, schedule & calendar views, user testing)

Repository: `https://github.com/yuqirichiechen/UWBSmartScheduler` (instructor/TA added as collaborator)

Deployed application: Vercel (public URL in repository README)

Project website: Hosted landing page (Vercel) — overview, architecture, and user guide

---

1. UW Community Impact Statement

(Rubric: UW Community Impact — 10 pts)

The problem

Every quarter, UW Bothell CSS students spend hours cross-referencing three disconnected systems to build a single schedule: **MyPlan** (degree requirements), the **Time Schedule** (live section times), and the **course catalog** (prerequisites and descriptions). These tools *display* data; none of them *help you decide*. A student with a real-life constraint — "I can't drive to campus on Fridays," "I work 30 hours a week," "I need to finish my CSS core but only have Tuesday/Thursday free" — has no intelligent tool that turns that sentence into a conflict-free, prerequisite-valid schedule.

Who it benefits and how

SmartScheduler serves **UW Bothell CSS students** (and is structurally extensible to other UWB programs). A student types their needs in plain English and receives a **section-level** recommendation — not just "take CSS 342," but "take CSS 342 **Section A**, TTh 2:00–3:20pm," because the system has verified that specific section fits every stated constraint and that the student has the prerequisites.

The community value is concrete and measurable:

- **Time saved:** collapses a multi-tab, multi-hour cross-referencing task into a single sentence and one click.
- **Fewer registration mistakes:** prerequisite enforcement prevents students from planning ineligible courses; conflict detection prevents double-booked sections before registration opens.
- **Accessibility:** the natural-language interface lowers the barrier for first-generation and non-traditional students who find the official tools opaque.
- **Forward planning:** the multi-quarter Year Planner lets students map an entire academic year (Autumn 2026 → Summer 2027) and see prerequisite chains across quarters.

Why this is the right problem

This is a documented, recurring pain point with no existing tool that closes the gap between *raw data* and *a decision*. The data changes every quarter, which makes a retrieval-augmented (RAG) approach — rather than a static or fine-tuned model — exactly the right architecture.

---

2. AI Integration Strategy

(Rubric: AI Integration — 15 pts. "AI is central to the application logic... advanced techniques or deep embedding. No 'side-chat' implementations.")

AI is **the core engine of the product, not a bolt-on chat box.** SmartScheduler is a **neuro-symbolic (hybrid) system**: a large language model performs the open-ended reasoning, and deterministic symbolic components guarantee correctness. Three AI techniques are embedded directly in the application's main path.

2.1 Retrieval-Augmented Generation (RAG) over the course catalog

We parsed the **official UW Bothell CSS course-descriptions catalog into 127 structured course records** (`css_catalog.json`) and embedded them into a **Google Gemini File Search store** using the `gemini-embedding-001` model. At query time, the relevant course documents are retrieved and supplied to **Gemini 2.5 Flash** as grounding context. This is classic RAG: embed → retrieve → generate, with the catalog as the knowledge base. The system supports two modes:

- **File Search mode** — a hosted vector store (fast, scalable, used in production).
- **Inline-context mode** — the compact catalog corpus is injected directly into the prompt (zero-setup fallback).

2.2 Natural-language constraint extraction (NLP)

A dedicated `ConstraintParser` converts plain-English requests into a structured constraint object: maximum/minimum credits, preferred days, avoided days, required courses, time-of-day windows, and in-person/online preference. It performs context-aware parsing — e.g., "I **work** afternoons" is correctly interpreted as *unavailability*, not a preference — which is precisely the kind of linguistic nuance that distinguishes meaningful NLP from keyword matching.

2.3 LLM-driven schedule selection, validated by symbolic reasoning

This is the heart of the "deep embedding." The LLM is given the student's request **and the real available sections**, and returns a **structured JSON selection** of `{course, section}` pairs with a rationale. The application then **validates every LLM pick against ground truth**:

- a **prerequisite graph** (transitive closure — completing CSS 343 implies CSS 342, 211, 161, 143) confirms eligibility;
- a deterministic **ConflictChecker** verifies no two selected sections overlap in time;
- a hydration step rejects any course the LLM hallucinated that isn't in the real offerings.

> ***Design principle: the LLM proposes, the code disposes.*** *The model handles the messy, human-language reasoning; the symbolic layer enforces hard guarantees (no conflicts, no missing prerequisites, credit caps respected). If the LLM is unavailable, returns invalid output, or proposes a conflicting set, the system* ***transparently falls back*** *to a fully deterministic schedule builder. The user always gets a correct, conflict-free schedule.*

This hybrid design is a recognized, advanced AI-engineering pattern (LLM reasoning + verifier) and is embedded in the application's primary control flow — the AI *is* the product, not a feature beside it.

---

3. Technical Execution & Specification

(Rubric: Technical Execution — 25 pts. "Well-structured, documented, stable/bug-free public deployment.")

3.1 Tech stack

Layer	Technology
Frontend	React 18 (SPA), Create React App, custom design system
Backend	Python 3, FastAPI (ASGI)
AI / RAG	Google Gemini 2.5 Flash + Gemini File Search (`gemini-embedding-001`)
Data	BeautifulSoup scraper + parsed catalog snapshot, SHA-256-validated cache
Hosting	Vercel (static frontend + Python serverless function, same origin)
Testing	pytest / custom eval harness (backend), React Testing Library + Jest (frontend)

3.2 Architecture & request flow

```
Browser (React SPA)
   ■  POST /api/schedule  { query, completed_courses }
   ▼
FastAPI (Vercel serverless function, api/index.py → backend/main.py)
   ■
   1. ConstraintParser    plain English → structured constraints
   2. Retrieval           vector search / catalog filter for relevant courses
   3. PrerequisiteGraph   expand completed courses via transitive closure
   4. GeminiRAG.select    LLM proposes {course, section} picks (structured JSON)
   5. Hydrate + Validate   map picks to REAL sections; ConflictChecker verifies
   6. Fallback            deterministic ScheduleBuilder if LLM invalid/unavailable
   ▼
   { recommendations, recommended_courses, is_valid, issues }
```

3.3 Backend components (all implemented)

- `UWScheduleScraper` — scrapes the public UW Bothell Time Schedule, with retry/back-off, an **SHA-256-validated cache**, **schema validation** (drops malformed rows and alerts on field-count deviation), and a sample-data fallback. Produces 30 live CSS courses with full section/meeting data.
- **Catalog pipeline** — `parse_catalog.py` converts the official descriptions page into **127 structured course records** (code, title, credits, gen-ed attributes, description, prerequisite text + parsed prerequisite codes, offered terms). This is the RAG knowledge base.
- `ConstraintParser` — natural-language → structured constraints (see §2.2).
- `PrerequisiteGraph` — directed prerequisite graph with transitive-closure inference.
- `ScheduleBuilder` — deterministic, conflict-aware, prerequisite-checked schedule generator (the verifier/fallback engine). Includes a time-window relaxation pass so an over-constrained request still returns the closest valid schedule rather than nothing.
- `ConflictChecker` — pairwise meeting-time overlap detection, credit-limit and avoided-day validation, prerequisite eligibility.
- `GeminiRAG` — Gemini File Search integration: `select_schedule()` (structured selection) and `recommend_schedule()` (grounded natural-language explanation).

3.4 Frontend (all implemented)

- **Schedule view** — natural-language query box with a **Claude-style clarifying prescreen**: when a request is open-ended ("what should I take?"), the app asks which courses the student has completed *before* generating, exactly as a human advisor would.

- **Catalog view** — searchable, expandable browser of all CSS courses with prerequisites, sections, and meeting times.
- **Calendar / Year Planner** — a multi-quarter manual planner (Autumn 2026 → Summer 2027) with per-quarter weekly calendar, section selection, live conflict detection, credit totals, and `localStorage` persistence.
- **Weekly calendar visualization** — color-coded section blocks rendered on a Mon–Fri grid with a legend.

3.5 Stability & quality assurance

- **Evaluation harness** (`backend/eval/run_eval.py`) automates the project's four success criteria and currently reports **10/10 conflict-free schedules, 0 prerequisite violations, deterministic scraper output across 3 runs, and 100% natural-language parsing accuracy** on the labeled test set.
- **Frontend test suite** (React Testing Library) — 5 passing tests covering API connection, schedule rendering, catalog loading, the clarifying-question flow, and year-planner navigation.
- **Graceful degradation** — the app is fully functional with no API keys (deterministic engine), and recovers transparently from LLM rate-limit/`503` errors.
- **Secrets hygiene** — API keys live only in gitignored `.env` / Vercel environment variables; GitHub push-protection is respected.

---

4. Project Web Presence

(Rubric: Project Web Presence — 15 pts)

The public-facing project website (hosted on Vercel) contains:

- **Project Overview** — the "why": the problem, the affected community, and the section-level differentiator, in professional, non-technical language.
- **Technical Documentation** — a high-level architecture diagram (the request-flow above), the tech-stack table, and a description of the neuro-symbolic AI design. Backed by `ARCHITECTURE.md` and `DEPLOY.md` in the repository.
- **User Guide** — step-by-step instructions for UW community members: how to phrase a request, how to add completed courses, how to read the weekly calendar, and how to use the multi-quarter planner. Example prompts are shown directly in the UI.

Design follows a clean, readable, paper-toned aesthetic with responsive layouts for desktop and mobile.

---

5. Milestone Roadmap & Planning

(Rubric: Milestones & Planning — 20 pts. "Adherence to timeline; iterative development; commit history reflects milestones; in-class milestone presentations + final presentation.")

Development followed iterative milestones, each reflected in the Git commit history and presented in class.

Milestone	Scope	Status
M1 — Data Pipeline	UW Bothell Time Schedule scraper; SHA-256-validated cache; schema validation; 127-course catalog parser	■ Complete
M2 — Core Logic (MVP)	ConstraintParser; PrerequisiteGraph; deterministic ScheduleBuilder; ConflictChecker; FastAPI endpoints	■ Complete

Milestone	Scope	Status
M3 — AI / RAG Integration	Gemini File Search store; RAG retrieval; LLM-driven structured selection; verifier + fallback	■ Complete
M4 — Frontend & UX	React SPA; schedule/catalog/calendar views; weekly-calendar visualization; clarifying-question prescreen	■ Complete
M5 — Testing & Evaluation	Automated eval harness (4 success criteria); frontend test suite; bug-fix passes	■ Complete
M6 — Deployment & Docs	Vercel deployment (static + serverless Python); project website; README/ARCHITECTURE/DEPLOY docs	■ Complete
Final Presentation	In-class demonstration of the deployed application	Scheduled

Evaluation criteria (measurable success metrics)

Test	Success criterion	Method	Result
10 student constraint queries	Zero schedule conflicts	Automated meeting-time verification	**10/10**
Prerequisite enforcement	No ineligible course recommended	Output vs. prerequisite-graph check	**0 violations**
Scraper reliability	Consistent output across runs	3 runs + hash comparison	**Identical hash**
Natural-language parsing	≥ 80% correct constraint extraction	Structured output vs. ground truth	**100%**

Risk mitigations (carried from pre-proposal, now implemented)

- **Scraper fragility** → schema assertions + cached snapshot + sample fallback.
- **Prerequisite gaps** → dedicated prerequisite graph layered on top of RAG output.
- **Schedule conflicts** → deterministic conflict-checker runs *after* the LLM, never inside it.
- **LLM unavailability / cost** → full deterministic fallback path.

---

6. MVP Scope

- CSS major courses (Time Schedule sections + 127-course catalog knowledge base).
- Day/time, credit-load, and required-course constraints.
- Prerequisite enforcement via a supplementary graph.
- Conflict detection before any recommendation is returned.
- Web interface: plain-English query input, structured schedule output, weekly calendar, course catalog, and multi-quarter planner.

Explicitly out of scope for the MVP: multi-major planning, real-time MyPlan write-back integration, and authenticated user-account persistence (the planner uses local browser storage instead).

---

7. Peer Review & Individual Contribution

(Rubric: Peer Review — 15 pts; submitted via confidential Canvas survey)

Each member will complete the confidential Canvas peer-review survey rating teammates (1–5) on **Technical Contribution, Reliability, Communication, and Problem Solving**, with qualitative examples and a self-reflection on authored modules.

Primary authorship (also visible in commit history):

- **Richie Chen** — scraper, catalog parser, caching/validation, embedding pipeline, FastAPI backend & deployment.
- **Kevin Vo** — Gemini RAG integration, prompt engineering, LLM-driven selection, conflict-detection logic.
- **Yousuf Al-Bassyiouni** — React frontend, schedule/catalog/calendar UI, weekly-calendar visualization, user testing.

Work was distributed equitably across the three members, with collaboration on integration points (API contracts, the AI-validation boundary) tracked through the shared repository.

Appendix — Repository & Deployment

- **Source:** `https://github.com/yuqirichiechen/UWBSmartScheduler` (instructor/TA added as collaborator).
- **Live app:** Vercel deployment (URL in README).
- **Key docs in repo:** `README.md`, `ARCHITECTURE.md`, `DEPLOY.md`, `TODO.md`, automated eval in `backend/eval/`.
- **Run locally:** backend `python main.py`; frontend `npm start`; evaluation `python -m eval.run_eval`.